

Informe de Análisis de Seguridad

Análisis estático de código fuente

Proyecto: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples

Fecha: 02/08/2026 14:29

Archivos analizados: 4

Total de hallazgos: 32

Duración del escaneo: 0.1 segundos

Severidad	Cantidad
■ Crítica	15
■ Alta	13
■ Media	4
■ Baja	0

Hallazgos de Seguridad

1. [CRÍTICA] Command Injection — <font color='Color(.862745,.078431,.235294,1)'+CWE-78

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\java\Vulnerable.java:22

Confianza: Media | **Lenguaje:** java

Descripción: Posible command injection: ejecución de comandos del sistema con datos de usuario sin sanitizar:
``Runtime.getRuntime().exec(``

Código:

```
public void runCommand(String host) { Runtime.getRuntime().exec("ping -c 1 " + host); }
```

Remediación:

Evita ejecutar comandos del sistema con input de usuario. Si es inevitable, usa listas de argumentos en vez de strings (ej: ``subprocess.run(['cmd', arg])`` en vez de ``os.system('cmd {arg}')`). JS: evita ``child_process.exec()`` con input de usuario. Usa ``execFile()`` con argumentos como array. PHP: evita ``exec()``, ``system()``, ``passthru()``, ``shell_exec()`` con input de usuario. Usa ``escapeshellarg()``. Java: evita ``Runtime.exec()`` con strings concatenados.

2. [CRÍTICA] Command Injection — <font color='Color(.862745,.078431,.235294,1)'+CWE-78

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\php\vulnerable.php:20

Confianza: Media | **Lenguaje:** php

Descripción: Posible command injection: ejecución de comandos del sistema con datos de usuario sin sanitizar:
``system("ping -c 1 " . $``

Código:

```
$host = $_GET['host']; $output = system("ping -c 1 " . $host); return $output;
```

Remediación:

Evita ejecutar comandos del sistema con input de usuario. Si es inevitable, usa listas de argumentos en vez de strings (ej: ``subprocess.run(['cmd', arg])`` en vez de ``os.system('cmd {arg}')`). JS: evita ``child_process.exec()`` con input de usuario. Usa ``execFile()`` con argumentos como array. PHP: evita ``exec()``, ``system()``, ``passthru()``, ``shell_exec()`` con input de usuario. Usa ``escapeshellarg()``. Java: evita ``Runtime.exec()`` con strings concatenados.

3. [CRÍTICA] Command Injection — <font color='Color(.862745,.078431,.235294,1)'+CWE-78

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\python\vulnerable.py:26

Confianza: Alta | **Lenguaje:** python

Descripción: Posible command injection: ejecución de comandos del sistema con datos de usuario sin sanitizar:
``subprocess.call("ping -c 1 " +``

Código:

```
host = request.GET.get("host") result = subprocess.call("ping -c 1 " + host, shell=True) return result
```

Remediación:

Evita ejecutar comandos del sistema con input de usuario. Si es inevitable, usa listas de argumentos en vez de strings (ej: ``subprocess.run(['cmd', arg])`` en vez de ``os.system('cmd {arg}')`). JS: evita ``child_process.exec()`` con input de usuario. Usa ``execFile()`` con argumentos como array. PHP: evita ``exec()``, ``system()``, ``passthru()``, ``shell_exec()`` con input de usuario. Usa ``escapeshellarg()``. Java: evita ``Runtime.exec()`` con strings concatenados.

4. [CRÍTICA] Hardcoded Secrets — <font color='Color(.862745,.078431,.235294,1)'+CWE-798

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\java\Vulnerable.java:7

Confianza: Media | **Lenguaje:** java

Descripción: Secreto hardcodeado detectado en el código fuente: ``API_KEY = "****REDACTED****"``

Código:

```
public class Vulnerable { private static final String API_KEY =  
"sk-1234567890abcdef1234567890abcdef"; private static final String DB_PASS = "admin123";
```

Remediación:

Nunca guardes secretos (API keys, contraseñas, tokens) en el código fuente. Usa variables de entorno (archivos .env) con librerías como python-dotenv. Usa servicios de gestión de secretos (AWS Secrets Manager, HashiCorp Vault). En CI/CD, inyecta los secretos como variables de entorno seguras. Revoca inmediatamente cualquier secreto que haya sido expuesto en el código.

5. [CRÍTICA] Hardcoded Secrets — **CWE-798**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\js\vulnerable.js:5

Confianza: Media | **Lenguaje:** javascript

Descripción: Secreto hardcodeado detectado en el código fuente: `API\_SECRET = "\*\*\*REDACTED\*\*\*"`

Código:

```
const API_SECRET = "super-secret-token-1234567890"; const DB_CONNECTION =  
"mongodb://admin:password123@localhost:27017/mydb";
```

Remediación:

Nunca guardes secretos (API keys, contraseñas, tokens) en el código fuente. Usa variables de entorno (archivos .env) con librerías como python-dotenv. Usa servicios de gestión de secretos (AWS Secrets Manager, HashiCorp Vault). En CI/CD, inyecta los secretos como variables de entorno seguras. Revoca inmediatamente cualquier secreto que haya sido expuesto en el código.

6. [CRÍTICA] Hardcoded Secrets — **CWE-798**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\php\vulnerable.php:3

Confianza: Media | **Lenguaje:** php

Descripción: Secreto hardcodeado detectado en el código fuente: `API\_KEY = "\*\*\*REDACTED\*\*\*"`

Código:

```
$API_KEY = "sk-1234567890abcdef1234567890abcdef"; $DB_PASSWORD = "supersecret123";
```

Remediación:

Nunca guardes secretos (API keys, contraseñas, tokens) en el código fuente. Usa variables de entorno (archivos .env) con librerías como python-dotenv. Usa servicios de gestión de secretos (AWS Secrets Manager, HashiCorp Vault). En CI/CD, inyecta los secretos como variables de entorno seguras. Revoca inmediatamente cualquier secreto que haya sido expuesto en el código.

7. [CRÍTICA] Hardcoded Secrets — **CWE-798**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\php\vulnerable.php:4

Confianza: Media | **Lenguaje:** php

Descripción: Secreto hardcodeado detectado en el código fuente: `PASSWORD = "\*\*\*REDACTED\*\*\*"`

Código:

```
$API_KEY = "sk-1234567890abcdef1234567890abcdef"; $DB_PASSWORD = "supersecret123";
```

Remediación:

Nunca guardes secretos (API keys, contraseñas, tokens) en el código fuente. Usa variables de entorno (archivos .env) con librerías como python-dotenv. Usa servicios de gestión de secretos (AWS Secrets Manager, HashiCorp Vault). En CI/CD, inyecta los secretos como variables de entorno seguras. Revoca inmediatamente cualquier secreto que haya sido expuesto en el código.

8. [CRÍTICA] Hardcoded Secrets — **CWE-798**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\python\vulnerable.py:6

Confianza: Media | **Lenguaje:** python

Descripción: Secreto hardcodeado detectado en el código fuente: `API\_KEY = "\*\*\*REDACTED\*\*\*"`

Código:

```
API_KEY = "sk-1234567890abcdef1234567890abcdef" DB_PASSWORD = "admin123"
```

Remediación:

Nunca guardes secretos (API keys, contraseñas, tokens) en el código fuente. Usa variables de entorno (archivos .env) con librerías como python-dotenv. Usa servicios de gestión de secretos (AWS Secrets Manager, HashiCorp Vault). En CI/CD, inyecta los secretos como variables de entorno seguras. Revoca inmediatamente cualquier secreto que haya sido expuesto en el código.

9. [CRÍTICA] Hardcoded Secrets — CWE-798

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\python\vulnerable.py:7

Confianza: Media | **Lenguaje:** python

Descripción: Secreto hardcodeado detectado en el código fuente: `PASSWORD = "\*\*\*REDACTED\*\*\*"`

Código:

```
API_KEY = "sk-1234567890abcdef1234567890abcdef" DB_PASSWORD = "admin123"
```

Remediación:

Nunca guardes secretos (API keys, contraseñas, tokens) en el código fuente. Usa variables de entorno (archivos .env) con librerías como python-dotenv. Usa servicios de gestión de secretos (AWS Secrets Manager, HashiCorp Vault). En CI/CD, inyecta los secretos como variables de entorno seguras. Revoca inmediatamente cualquier secreto que haya sido expuesto en el código.

10. [CRÍTICA] Sql Injection — CWE-89

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\java\Vulnerable.java:11

Confianza: Media | **Lenguaje:** java

Descripción: Posible SQL injection detectada: ejecución de consulta SQL con concatenación de variables: `SELECT \* FROM users WHERE id = "" + userId + ""`;

Código:

```
public String getUser(String userId) { String query = "SELECT * FROM users WHERE id = '" + userId + "'"; Statement stmt = connection.createStatement();
```

Remediación:

Usa consultas parametrizadas (prepared statements) en lugar de concatenar strings. Python: usa parámetros con `cursor.execute(query, params)`. JS: usa placeholders con `db.query("SELECT \* FROM users WHERE id = ?", [userId])`. PHP: usa PDO con prepared statements. Java: usa PreparedStatement en JDBC. Si debes construir queries dinámicas, sanitiza y escapa todas las entradas.

11. [CRÍTICA] Sql Injection — CWE-89

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\js\vulnerable.js:10

Confianza: Alta | **Lenguaje:** javascript

Descripción: Posible SQL injection detectada: ejecución de consulta SQL con concatenación de variables: `SELECT \* FROM users WHERE id = "" + userId + ""`;

Código:

```
const userId = req.query.id; const query = "SELECT * FROM users WHERE id = '" + userId + "'"; db.query(query);
```

Remediación:

Usa consultas parametrizadas (prepared statements) en lugar de concatenar strings. Python: usa parámetros con `cursor.execute(query, params)`. JS: usa placeholders con `db.query("SELECT \* FROM users WHERE id = ?", [userId])`. PHP: usa PDO con prepared statements. Java: usa PreparedStatement en JDBC. Si debes construir queries dinámicas, sanitiza y escapa todas las entradas.

12. [CRÍTICA] Sql Injection — CWE-89

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\php\vulnerable.php:8

Confianza: Media | **Lenguaje:** php

Descripción: Posible SQL injection detectada: ejecución de consulta SQL con concatenación de variables: `SELECT \* FROM users WHERE id = "" . \$id . ""`;

Código:

```
$id = $_GET['id']; $query = "SELECT * FROM users WHERE id = '' . $id . ''"; $result = mysqli_query($conn, $query);
```

Remediación:

Usa consultas parametrizadas (prepared statements) en lugar de concatenar strings. Python: usa parámetros con `cursor.execute(query, params)`. JS: usa placeholders con `db.query('SELECT \* FROM users WHERE id = ?', [userId])`. PHP: usa PDO con prepared statements. Java: usa PreparedStatement en JDBC. Si debes construir queries dinámicas, sanitiza y escapa todas las entradas.

13. [CRÍTICA] Sql Injection — CWE-89

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\php\vulnerable.php:9

Confianza: Media | **Lenguaje:** php

Descripción: Posible SQL injection detectada: ejecución de consulta SQL con concatenación de variables: `mysqli\_query(\$conn, \$`

Código:

```
$query = "SELECT * FROM users WHERE id = '' . $id . ''"; $result = mysqli_query($conn, $query); return $result;
```

Remediación:

Usa consultas parametrizadas (prepared statements) en lugar de concatenar strings. Python: usa parámetros con `cursor.execute(query, params)`. JS: usa placeholders con `db.query('SELECT \* FROM users WHERE id = ?', [userId])`. PHP: usa PDO con prepared statements. Java: usa PreparedStatement en JDBC. Si debes construir queries dinámicas, sanitiza y escapa todas las entradas.

14. [CRÍTICA] Sql Injection — CWE-89

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\python\vulnerable.py:13

Confianza: Alta | **Lenguaje:** python

Descripción: Posible SQL injection detectada: ejecución de consulta SQL con concatenación de variables: `SELECT \* FROM users WHERE user="" + user + "" AND pass="" + passwd + ""`

Código:

```
passwd = request.GET.get("pass") query = "SELECT * FROM users WHERE user='" + user + "' AND pass='" + passwd + "'" cursor.execute(query)
```

Remediación:

Usa consultas parametrizadas (prepared statements) en lugar de concatenar strings. Python: usa parámetros con `cursor.execute(query, params)`. JS: usa placeholders con `db.query('SELECT \* FROM users WHERE id = ?', [userId])`. PHP: usa PDO con prepared statements. Java: usa PreparedStatement en JDBC. Si debes construir queries dinámicas, sanitiza y escapa todas las entradas.

15. [CRÍTICA] Sql Injection — CWE-89

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\python\vulnerable.py:14

Confianza: Alta | **Lenguaje:** python

Descripción: Posible SQL injection detectada: ejecución de consulta SQL con concatenación de variables: `cursor.execute()` recibe datos de usuario no sanitizados

Código:

```
query = "SELECT * FROM users WHERE user='" + user + "' AND pass='" + passwd + "'" cursor.execute(query) return cursor.fetchone()
```

Remediación:

Usa consultas parametrizadas (prepared statements) en lugar de concatenar strings. Python: usa parámetros con ``cursor.execute(query, params)``. JS: usa placeholders con ``db.query('SELECT * FROM users WHERE id = ?', [userId])``. PHP: usa PDO con prepared statements. Java: usa PreparedStatement en JDBC. Si debes construir queries dinámicas, sanitiza y escapa todas las entradas.

16. [ALTA] Dynamic Exec — **CWE-95**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\java\Vulnerable.java:36

Confianza: Media | **Lenguaje:** java

Descripción: Ejecución dinámica de código detectada (eval/exec/Function): ``eval(``

Código:

```
ScriptEngine engine = new ScriptEngineManager().getEngineByName("JavaScript"); engine.eval(code);
}
```

Remediación:

Evita ``eval()``, ``exec()``, ``Function()`` con datos dinámicos. Si necesitas ejecutar código dinámico, usa sandboxing o alternativas seguras. Python: evita ``eval()`` y ``exec()``. Si necesitas evaluar expresiones simples, usa ``ast.literal_eval()``. JS/TS: no uses ``eval()`` ni ``new Function()``. Para parsear JSON usa ``JSON.parse()``. PHP: evita ``eval()``, ``preg_replace()`` con modificador /e, ``create_function()``. La ejecución de código en tiempo de ejecución casi siempre se puede evitar rediseñando la lógica.

17. [ALTA] Dynamic Exec — **CWE-95**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\java\Vulnerable.java:22

Confianza: Media | **Lenguaje:** java

Descripción: Ejecución dinámica de código detectada (eval/exec/Function): ``exec(``

Código:

```
public void runCommand(String host) { Runtime.getRuntime().exec("ping -c 1 " + host); }
```

Remediación:

Evita ``eval()``, ``exec()``, ``Function()`` con datos dinámicos. Si necesitas ejecutar código dinámico, usa sandboxing o alternativas seguras. Python: evita ``eval()`` y ``exec()``. Si necesitas evaluar expresiones simples, usa ``ast.literal_eval()``. JS/TS: no uses ``eval()`` ni ``new Function()``. Para parsear JSON usa ``JSON.parse()``. PHP: evita ``eval()``, ``preg_replace()`` con modificador /e, ``create_function()``. La ejecución de código en tiempo de ejecución casi siempre se puede evitar rediseñando la lógica.

18. [ALTA] Dynamic Exec — **CWE-95**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\js\vulnerable.js:24

Confianza: Alta | **Lenguaje:** javascript

Descripción: Ejecución dinámica de código detectada (eval/exec/Function): ``eval(``

Código:

```
const code = req.query.code; eval("var user = " + code + ";"); res.json(user);
```

Remediación:

Evita ``eval()``, ``exec()``, ``Function()`` con datos dinámicos. Si necesitas ejecutar código dinámico, usa sandboxing o alternativas seguras. Python: evita ``eval()`` y ``exec()``. Si necesitas evaluar expresiones simples, usa ``ast.literal_eval()``. JS/TS: no uses ``eval()`` ni ``new Function()``. Para parsear JSON usa ``JSON.parse()``. PHP: evita ``eval()``, ``preg_replace()`` con modificador /e, ``create_function()``. La ejecución de código en tiempo de ejecución casi siempre se puede evitar rediseñando la lógica.

19. [ALTA] Dynamic Exec — **CWE-95**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\js\vulnerable.js:30

Confianza: Media | **Lenguaje:** javascript

Descripción: Ejecución dinámica de código detectada (eval/exec/Function): `exec`

Código:

```
const host = req.query.host; const child = require("child_process").exec("ping -c 1 " + host); });
```

Remediación:

Evita `eval()`, `exec()`, `Function()` con datos dinámicos. Si necesitas ejecutar código dinámico, usa sandboxing o alternativas seguras. Python: evita `eval()` y `exec()`. Si necesitas evaluar expresiones simples, usa `ast.literal\_eval()`. JS/TS: no uses `eval()` ni `new Function()`. Para parsear JSON usa `JSON.parse()`. PHP: evita `eval()`, `preg\_replace()` con modificador /e, `create\_function()`. La ejecución de código en tiempo de ejecución casi siempre se puede evitar rediseñando la lógica.

20. [ALTA] Dynamic Exec — **CWE-95**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\php\vulnerable.php:37

Confianza: Media | **Lenguaje:** php

Descripción: Ejecución dinámica de código detectada (eval/exec/Function): `eval`

Código:

```
$code = $_GET['code']; eval($code); }
```

Remediación:

Evita `eval()`, `exec()`, `Function()` con datos dinámicos. Si necesitas ejecutar código dinámico, usa sandboxing o alternativas seguras. Python: evita `eval()` y `exec()`. Si necesitas evaluar expresiones simples, usa `ast.literal\_eval()`. JS/TS: no uses `eval()` ni `new Function()`. Para parsear JSON usa `JSON.parse()`. PHP: evita `eval()`, `preg\_replace()` con modificador /e, `create\_function()`. La ejecución de código en tiempo de ejecución casi siempre se puede evitar rediseñando la lógica.

21. [ALTA] Insecure Deserialization — **CWE-502**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\java\Vulnerable.java:30

Confianza: Media | **Lenguaje:** java

Descripción: Deserialización de datos no confiables detectada: `ObjectInputStream`

Código:

```
public Object restoreState(byte[] data) throws Exception { ObjectInputStream ois = new ObjectInputStream(new ByteArrayInputStream(data)); return ois.readObject(); }
```

Remediación:

No deserialices datos que provengan de fuentes no confiables (usuarios, red). Python: evita `pickle.loads()` con datos externos. Usa JSON para serialización segura. JS: ten cuidado con `JSON.parse()` cuando los datos no están validados; no uses `eval()`. PHP: evita `unserialize()` con datos de usuario. Usa JSON. Java: usa whitelists de clases con `ObjectInputStream`, o evita la deserialización de Java nativo. Prefiere formatos de datos seguros como JSON, Protocol Buffers, o MessagePack.

22. [ALTA] Insecure Deserialization — **CWE-502**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\php\vulnerable.php:31

Confianza: Media | **Lenguaje:** php

Descripción: Deserialización de datos no confiables detectada: `unserialize`

Código:

```
function loadUserData($data) { $user = unserialize($data); return $user; }
```

Remediación:

No deserialices datos que provengan de fuentes no confiables (usuarios, red). Python: evita `pickle.loads()` con datos externos. Usa JSON para serialización segura. JS: ten cuidado con `JSON.parse()` cuando los datos no están validados; no uses `eval()`. PHP: evita `unserialize()` con datos de usuario. Usa JSON. Java: usa whitelists de clases con `ObjectInputStream`, o evita la deserialización de Java nativo. Prefiere formatos de datos seguros como JSON, Protocol Buffers, o MessagePack.

23. [ALTA] Insecure Deserialization — **CWE-502**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\python\vulnerable.py:32

Confianza: Media | **Lenguaje:** python

Descripción: Deserialización de datos no confiables detectada: ``pickle.loads(``

Código:

```
try: return pickle.loads(data) except Exception:
```

Remediación:

No deserialices datos que provengan de fuentes no confiables (usuarios, red).Python: evita ``pickle.loads(`` con datos externos. Usa JSON para serialización segura.JS: ten cuidado con ``JSON.parse(`` cuando los datos no están validados; no uses ``eval(``.PHP: evita ``unserialize(`` con datos de usuario. Usa JSON.Java: usa whitelists de clases con ``ObjectInputStream`,` o evita la deserialización de Java nativo.Prefiere formatos de datos seguros como JSON, Protocol Buffers, o MessagePack.

24. [ALTA] Path Traversal — **CWE-22**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\python\vulnerable.py:32

Confianza: Alta | **Lenguaje:** python

Descripción: Posible path traversal: construcción de rutas de archivo con input de usuario: ``open(`` recibe datos de usuario no sanitizados

Código:

```
path = os.path.join("/var/www/uploads", filename) return open(path, "rb").read()
```

Remediación:

No uses input de usuario directamente para construir rutas de archivo.Usa ``os.path.basename(`` para sanitizar nombres de archivo.Normaliza y valida la ruta resultante contra un directorio base permitido.Python: ``os.path.realpath(`` y verifica que esté dentro del directorio permitido.JS: ``path.resolve(`` y verifica que no escape del directorio base.Usa whitelists de nombres de archivo permitidos en lugar de blacklists de caracteres peligrosos.

25. [ALTA] Ssrif — **CWE-918**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\js\vulnerable.js:39

Confianza: Alta | **Lenguaje:** javascript

Descripción: Posible SSRF: petición HTTP a URL controlada por el usuario: ``fetch(`` recibe datos de usuario no sanitizados

Código:

```
const url = req.query.url; return fetch(url); };
```

Remediación:

No permitas que los usuarios especifiquen URLs arbitrarias para peticiones del servidor.Implementa una whitelist de dominios/hosts permitidos.Bloquea accesos a IPs internas (127.0.0.1, 10.x, 172.16-31.x, 192.168.x).Valida que la URL apunte solo a protocolos http/https, no file://, gopher://, etc.Python: usa una lista de hosts permitidos con ``urllib.parse.urlparse(``.JS: valida la URL antes de pasarla a ``axios`, `fetch`, `http.request`.`

26. [ALTA] Tainted Sink — **CWE-20**

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\js\vulnerable.js:12

Confianza: Alta | **Lenguaje:** javascript

Descripción: Flujo de datos de usuario detectado hacia función peligrosa ``res.send(``

Código:

```
db.query(query); res.send("<div>Hello " + userId + "</div>"); };
```

Remediación:

Valida y sanitiza los datos de usuario antes de pasarlos a funciones peligrosas.

27. [ALTA] Xss — CWE-79

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\js\vulnerable.js:19

Confianza: Media | **Lenguaje:** javascript

Descripción: Posible XSS (Cross-Site Scripting) detectada: renderizado de contenido no sanitizado en el DOM: ``innerHTML =``

Código:

```
const html = '<div>welcome <b>' + user + '</b></div>';
document.getElementById("content").innerHTML = html; });
```

Remediación:

Escapa siempre el contenido antes de insertarlo en el DOM.JS: no uses ``innerHTML`` / ``dangerouslySetInnerHTML`` con datos de usuario. Usa ``textContent`` o sanitización con `DOMPurify`. React: evita ``dangerouslySetInnerHTML`` a menos que sanitices con `DOMPurify`. Python templates: usa `autoescape` (Django, Jinja2). PHP: usa ``htmlspecialchars()`` al renderizar en HTML. Valida y sanitiza cualquier input de usuario antes de mostrarlo.

28. [ALTA] Xss — CWE-79

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\js\vulnerable.js:24

Confianza: Media | **Lenguaje:** javascript

Descripción: Posible XSS (Cross-Site Scripting) detectada: renderizado de contenido no sanitizado en el DOM: ``eval("var user = " + code +``

Código:

```
const code = req.query.code; eval("var user = " + code + ";"); res.json(user);
```

Remediación:

Escapa siempre el contenido antes de insertarlo en el DOM.JS: no uses ``innerHTML`` / ``dangerouslySetInnerHTML`` con datos de usuario. Usa ``textContent`` o sanitización con `DOMPurify`. React: evita ``dangerouslySetInnerHTML`` a menos que sanitices con `DOMPurify`. Python templates: usa `autoescape` (Django, Jinja2). PHP: usa ``htmlspecialchars()`` al renderizar en HTML. Valida y sanitiza cualquier input de usuario antes de mostrarlo.

29. [MEDIA] Insecure Crypto — CWE-327

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\java\Vulnerable.java:45

Confianza: Media | **Lenguaje:** java

Descripción: Uso de algoritmo criptográfico débil u obsoleto detectado: ``MessageDigest.getInstance("MD5")``

Código:

```
public String hashPassword(String pass) throws Exception { MessageDigest md =
MessageDigest.getInstance("MD5"); byte[] digest = md.digest(pass.getBytes());
```

Remediación:

No uses MD5 o SHA1 para hashing de contraseñas o firmas. Para contraseñas: usa `bcrypt`, `scrypt`, `argon2`, o `PBKDF2` con salt. Para integridad: usa `SHA-256` o `SHA-3`. Python: usa ``hashlib.sha256()`` en vez de ``hashlib.md5()``, ``bcrypt`` para contraseñas. JS: usa ``crypto.createHash('sha256')`` en vez de ``md5``. No implementes criptografía propia, usa librerías estándar y auditadas.

30. [MEDIA] Insecure Crypto — CWE-327

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\js\vulnerable.js:34

Confianza: Media | **Lenguaje:** javascript

Descripción: Uso de algoritmo criptográfico débil u obsoleto detectado: ``crypto.createHash("md5")``

Código:

```
function hashPassword(password) { return crypto.createHash("md5").update(password).digest("hex");
}
```

Remediación:

No uses MD5 o SHA1 para hashing de contraseñas o firmas. Para contraseñas: usa bcrypt, scrypt, argon2, o PBKDF2 con salt. Para integridad: usa SHA-256 o SHA-3. Python: usa `hashlib.sha256()` en vez de `hashlib.md5()`, `bcrypt` para contraseñas. JS: usa `crypto.createHash('sha256')` en vez de `md5`. No implementes criptografía propia, usa librerías estándar y auditadas.

31. [MEDIA] Insecure Crypto — CWE-327

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\php\vulnerable.php:47

Confianza: Media | **Lenguaje:** php

Descripción: Uso de algoritmo criptográfico débil u obsoleto detectado: `md5`

Código:

```
function hashPass($password) { return md5($password); }
```

Remediación:

No uses MD5 o SHA1 para hashing de contraseñas o firmas. Para contraseñas: usa bcrypt, scrypt, argon2, o PBKDF2 con salt. Para integridad: usa SHA-256 o SHA-3. Python: usa `hashlib.sha256()` en vez de `hashlib.md5()`, `bcrypt` para contraseñas. JS: usa `crypto.createHash('sha256')` en vez de `md5`. No implementes criptografía propia, usa librerías estándar y auditadas.

32. [MEDIA] Insecure Crypto — CWE-327

Archivo: C:\Users\oscar\Desktop\Proyectos\análisis_código\tests\samples\python\vulnerable.py:38

Confianza: Media | **Lenguaje:** python

Descripción: Uso de algoritmo criptográfico débil u obsoleto detectado: `hashlib.md5`

Código:

```
def hash_password(passwd): return hashlib.md5(passwd.encode()).hexdigest()
```

Remediación:

No uses MD5 o SHA1 para hashing de contraseñas o firmas. Para contraseñas: usa bcrypt, scrypt, argon2, o PBKDF2 con salt. Para integridad: usa SHA-256 o SHA-3. Python: usa `hashlib.sha256()` en vez de `hashlib.md5()`, `bcrypt` para contraseñas. JS: usa `crypto.createHash('sha256')` en vez de `md5`. No implementes criptografía propia, usa librerías estándar y auditadas.
